

Module 1 Homework: Chapter 2 Exercises, Problems 8, 9, and 10

Ryan Waterman, 7/8/25

Problem 8

This exercise relates to the College data set, which can be found in the file College.csv on the book website. It contains a number of variables for 777 different universities and colleges in the US. The variables are:

- Private : Public/private indicator
- Apps : Number of applications received
- Accept : Number of applicants accepted
- Enroll : Number of new students enrolled
- Top10perc : New students from top 10% of high school class
- Top25perc : New students from top 25% of high school class
- F.Undergrad : Number of full-time undergraduates
- P.Undergrad : Number of part-time undergraduates
- Outstate : Out-of-state tuition
- Room.Board : Room and board costs
- Books : Estimated book costs
- Personal : Estimated personal spending
- PhD : Percent of faculty with Ph.D.s
- Terminal : Percent of faculty with terminal degree
- S.F.Ratio : Student/faculty ratio
- perc.alumni : Percent of alumni who donate
- Expend : Instructional expenditure per student
- Grad.Rate : Graduation rate

Before reading the data into Python, it can be viewed in Excel or a text editor.

Part A

Use the `pd.read_csv()` function to read the data into Python. Call the loaded data `college`. Make sure that you have the directory set to the correct location for the data.

```
In [2]: import pandas as pd
```

```
In [3]: data_path = './data/College.csv'
```

```
In [4]: college = pd.read_csv(data_path)
college.head(5)
```

```
Out[4]:
```

	Unnamed: 0	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad
0	Abilene Christian University	Yes	1660	1232	721	23	52	2885	
1	Adelphi University	Yes	2186	1924	512	16	29	2683	
2	Adrian College	Yes	1428	1097	336	22	50	1036	
3	Agnes Scott College	Yes	417	349	137	60	89	510	
4	Alaska Pacific University	Yes	193	146	55	16	44	249	

Part B

Look at the data used in the notebook by creating and running a new cell with just the code `college` in it. You should notice that the first column is just the name of each university in a column named something like `Unnamed: 0`. We don't really want pandas to treat this as data. However, it may be handy to have these names for later. Try the following commands and similarly look at the resulting data frames:

```
college2 = pd.read_csv('College.csv', index_col=0)
college3 = college.rename({'Unnamed: 0': 'College'},
axis=1)
college3 = college3.set_index('College')
```

This has used the first column in the file as an index for the data frame. This means that pandas has given each row a name corresponding to the appropriate university. Now you should see that the first data column is `Private`. Note that the names of the colleges appear on the left of the table. We also introduced a new python object above: a dictionary, which is specified by dictionary(key, value) pairs. Keep your modified version of the data with the following:

```
college = college3
```

```
In [5]: college2 = pd.read_csv(data_path, index_col=0)
college2.head(5)
```

Out[5]:

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad
Abilene Christian University	Yes	1660	1232	721	23	52	2885	537
Adelphi University	Yes	2186	1924	512	16	29	2683	1227
Adrian College	Yes	1428	1097	336	22	50	1036	99
Agnes Scott College	Yes	417	349	137	60	89	510	65
Alaska Pacific University	Yes	193	146	55	16	44	249	869

In [6]: `college3 = college.rename({'Unnamed: 0': 'College'}, axis = 1)`
`college3.head(5)`

Out[6]:

	College	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad
0	Abilene Christian University	Yes	1660	1232	721	23	52	2885	537
1	Adelphi University	Yes	2186	1924	512	16	29	2683	1227
2	Adrian College	Yes	1428	1097	336	22	50	1036	99
3	Agnes Scott College	Yes	417	349	137	60	89	510	65
4	Alaska Pacific University	Yes	193	146	55	16	44	249	869

In [7]: `college3 = college3.set_index('College')`
`college3.head(5)`

Out[7]:

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad
College								
Abilene Christian University	Yes	1660	1232	721	23	52	2885	537
Adelphi University	Yes	2186	1924	512	16	29	2683	1227
Adrian College	Yes	1428	1097	336	22	50	1036	99
Agnes Scott College	Yes	417	349	137	60	89	510	63
Alaska Pacific University	Yes	193	146	55	16	44	249	869

In [8]: `college = college3`
`college.head(5)`

Out[8]:

	Private	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.Undergrad
College								
Abilene Christian University	Yes	1660	1232	721	23	52	2885	537
Adelphi University	Yes	2186	1924	512	16	29	2683	1227
Adrian College	Yes	1428	1097	336	22	50	1036	99
Agnes Scott College	Yes	417	349	137	60	89	510	63
Alaska Pacific University	Yes	193	146	55	16	44	249	869

Part C

Use the `describe()` method of to produce a numerical summary of the variables in the data set.

In [9]: `college.describe()`

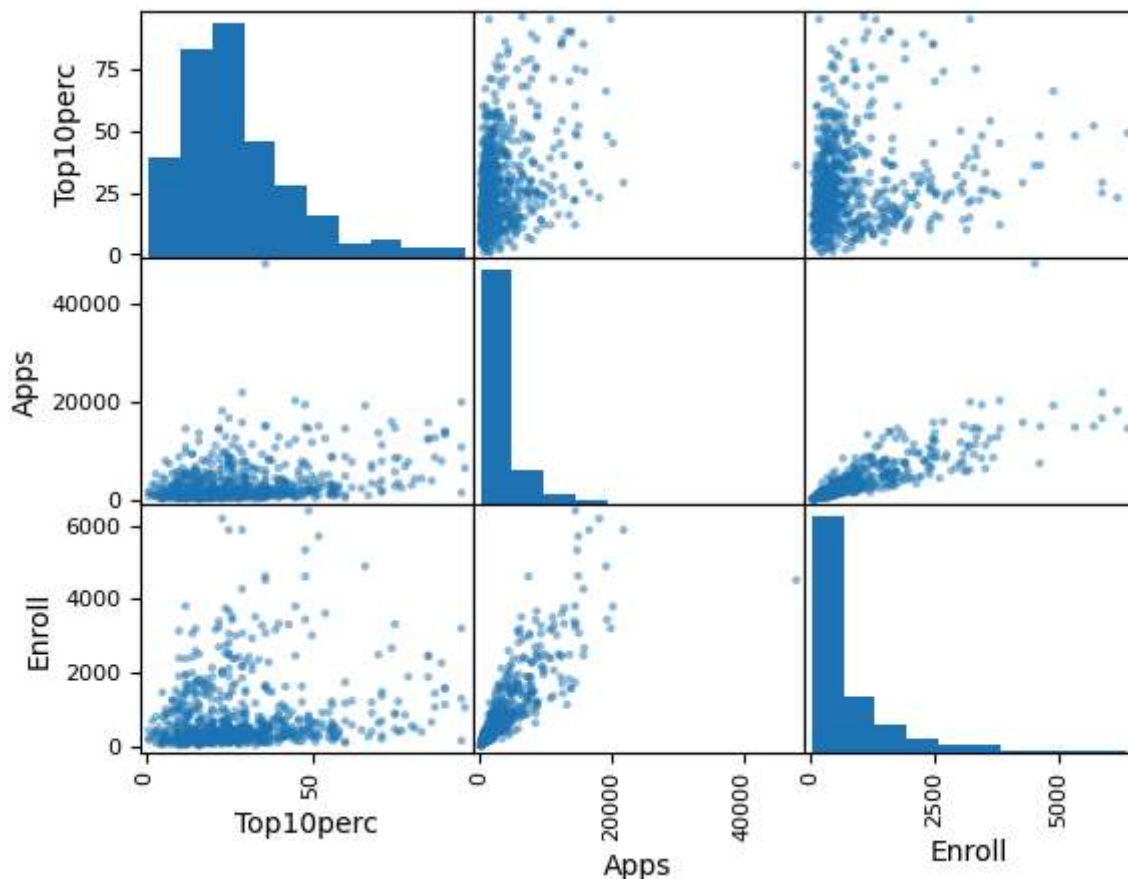
Out[9]:

	Apps	Accept	Enroll	Top10perc	Top25perc	F.Undergrad	P.U
count	777.000000	777.000000	777.000000	777.000000	777.000000	777.000000	7
mean	3001.638353	2018.804376	779.972973	27.558559	55.796654	3699.907336	8
std	3870.201484	2451.113971	929.176190	17.640364	19.804778	4850.420531	15
min	81.000000	72.000000	35.000000	1.000000	9.000000	139.000000	
25%	776.000000	604.000000	242.000000	15.000000	41.000000	992.000000	
50%	1558.000000	1110.000000	434.000000	23.000000	54.000000	1707.000000	3
75%	3624.000000	2424.000000	902.000000	35.000000	69.000000	4005.000000	9
max	48094.000000	26330.000000	6392.000000	96.000000	100.000000	31643.000000	218

Part D

Use the `pd.plotting.scatter_matrix()` function to produce a scatterplot matrix of the first columns [Top10perc, Apps, Enroll]. Recall that you can reference a list C of columns of a data frame A using `A[C]`.

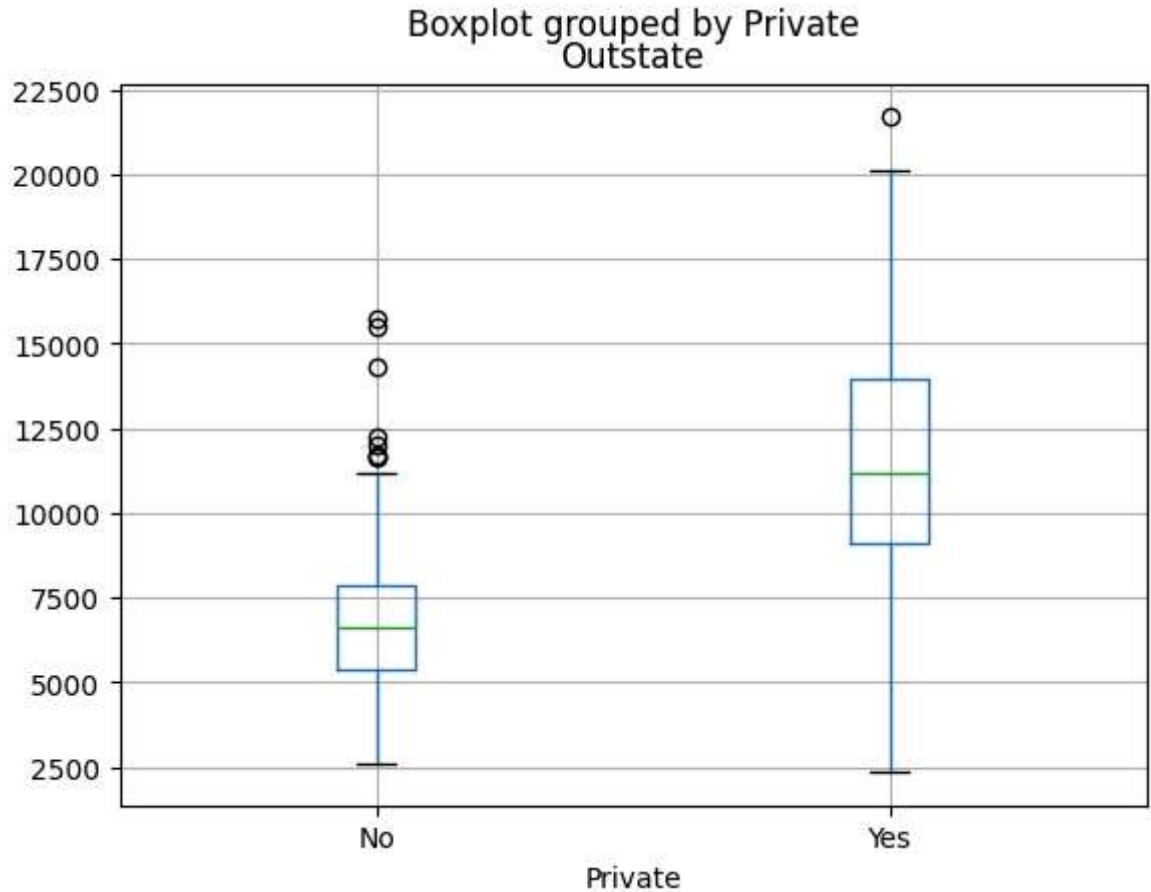
```
In [10]: pd.plotting.scatter_matrix(college[['Top10perc', 'Apps', 'Enroll']]);
```



Part E

Use the `boxplot()` method of `college` to produce side-by-side boxplots of `Outstate` versus `Private`.

```
In [11]: college.boxplot('Outstate', 'Private');
```



Part F

Create a new qualitative variable, called `Elite`, by binning the `Top10perc` variable into two groups based on whether or not the proportion of students coming from the top 10% of their high school classes exceeds 50%.

```
college['Elite'] = pd.cut(college['Top10perc'],
                          [0,0.5,1],
                          labels=['No', 'Yes'])
```

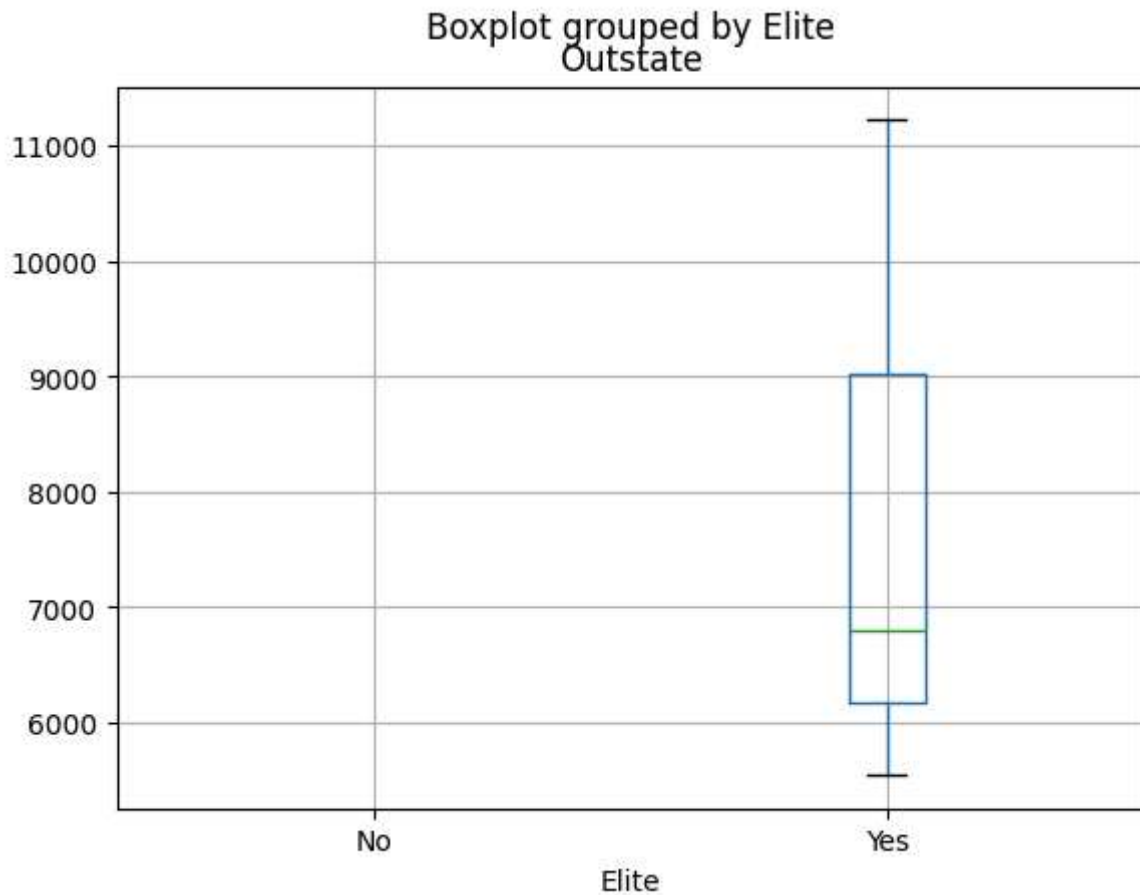
Use the `value_counts()` method of `college['Elite']` to see how many elite universities there are. Finally, use the `boxplot()` method again to produce side-by-side boxplots of `Outstate` versus `Elite`.

```
In [12]: college['Elite'] = pd.cut(college['Top10perc'], [0,0.5,1], labels=['No', 'Yes'])
```

```
In [13]: college['Elite'].value_counts()
```

```
Out[13]: Elite
Yes      3
No       0
Name: count, dtype: int64
```

```
In [14]: college.boxplot('Outstate', 'Elite');
```



```
In [15]: import matplotlib.pyplot as plt
```

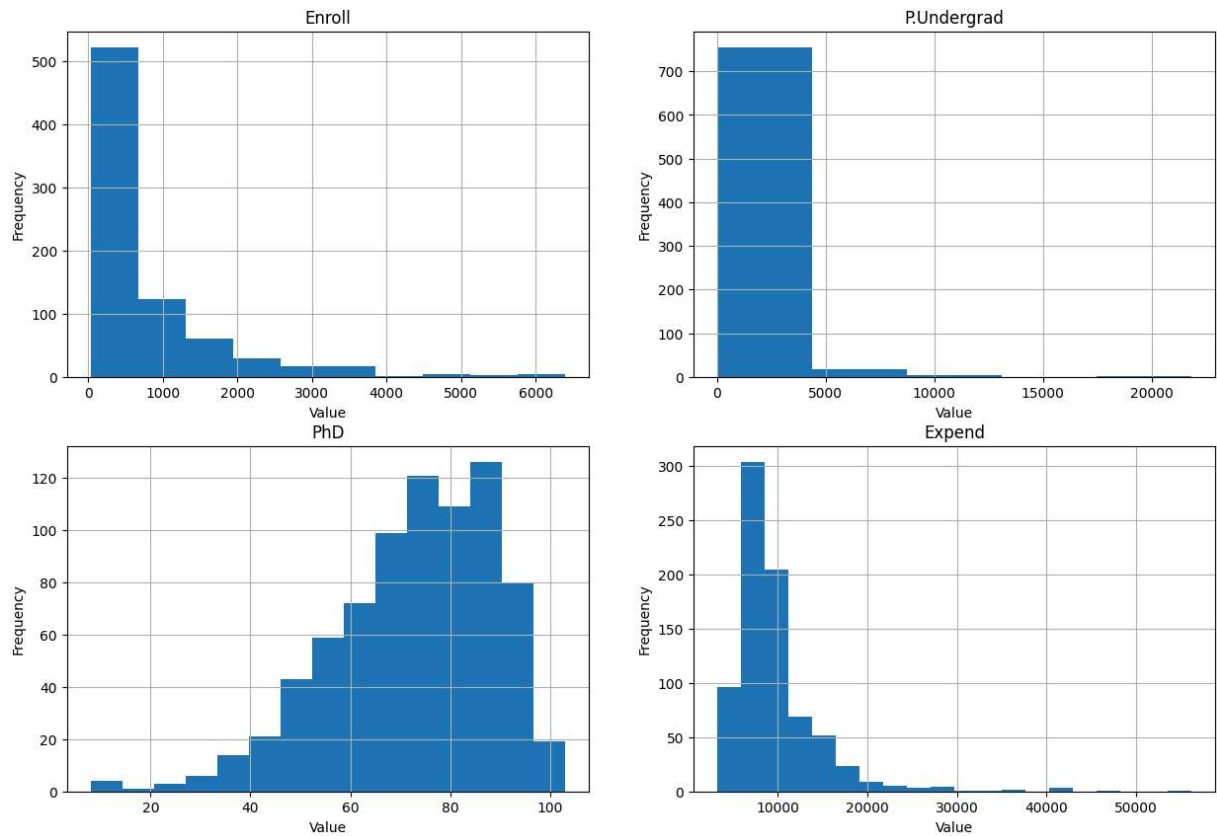
```
In [16]: fig, axes = plt.subplots(2, 2, figsize=(15,10));

college['Enroll'].hist(ax=axes[0,0],bins = 10);
axes[0,0].set_title('Enroll');
axes[0,0].set_xlabel('Value');
axes[0,0].set_ylabel('Frequency');

college['P.Undergrad'].hist(ax=axes[0,1],bins = 5);
axes[0,1].set_title('P.Undergrad');
axes[0,1].set_xlabel('Value');
axes[0,1].set_ylabel('Frequency');

college['PhD'].hist(ax=axes[1,0],bins = 15);
axes[1,0].set_title('PhD');
axes[1,0].set_xlabel('Value');
axes[1,0].set_ylabel('Frequency');
```

```
college['Expend'].hist(ax=axes[1,1],bins = 20);
axes[1,1].set_title('Expend');
axes[1,1].set_xlabel('Value');
axes[1,1].set_ylabel('Frequency');
```

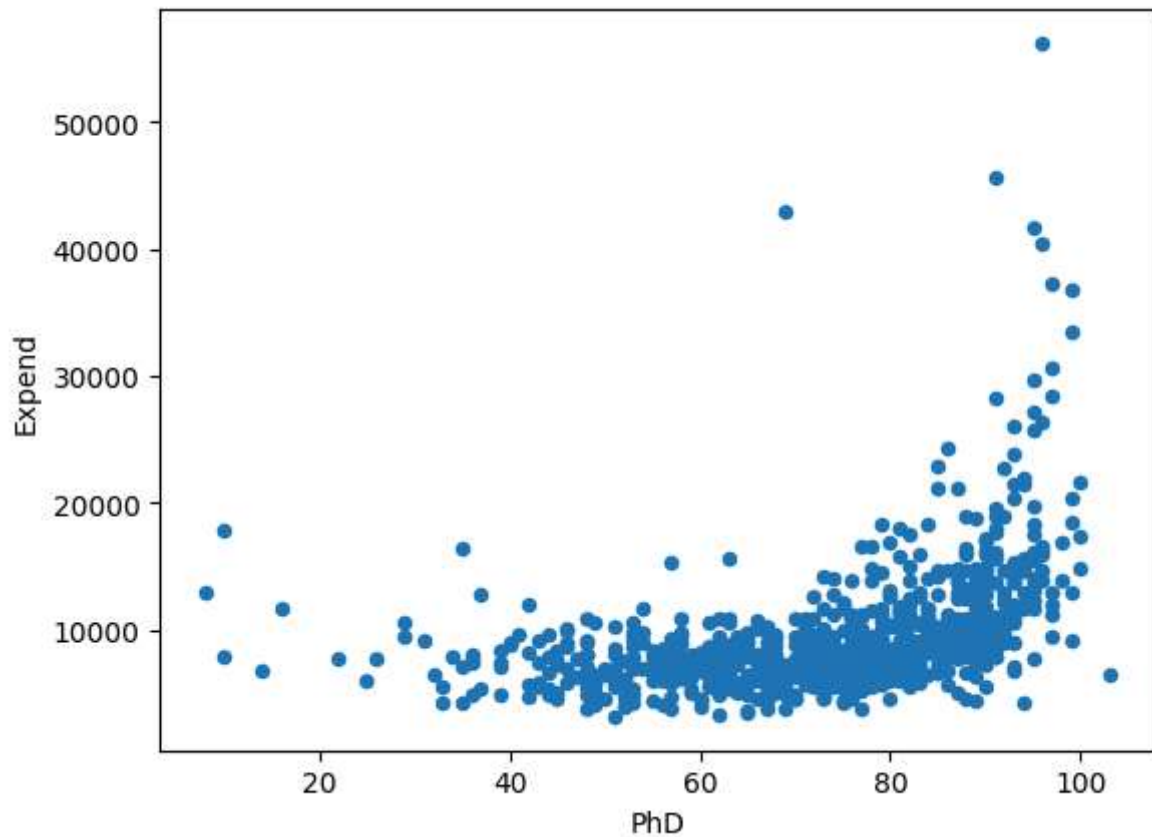


Part H

Continue exploring the data, and provide a brief summary of what you discover.

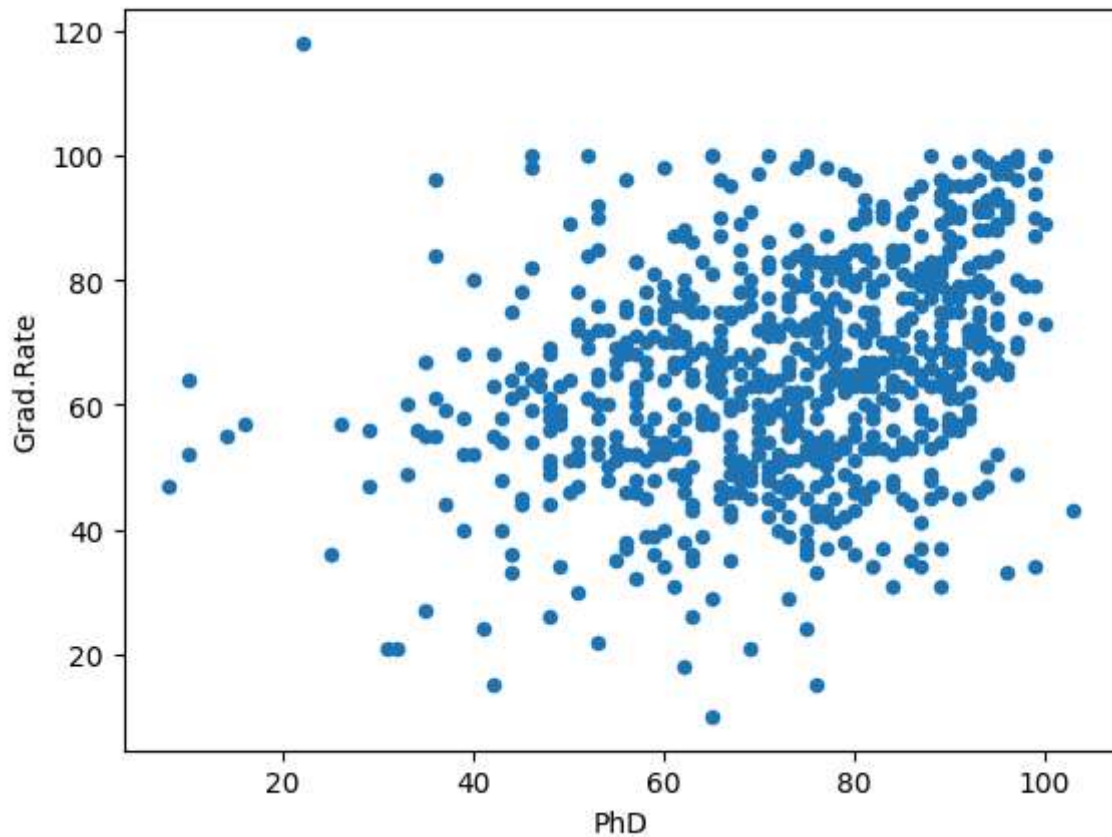
I am particularly interested in the relationship between expenditure and percentage of faculty with PhDs. Let's explore that relationship.

```
In [17]: college.plot.scatter(x='PhD', y='Expend');
```



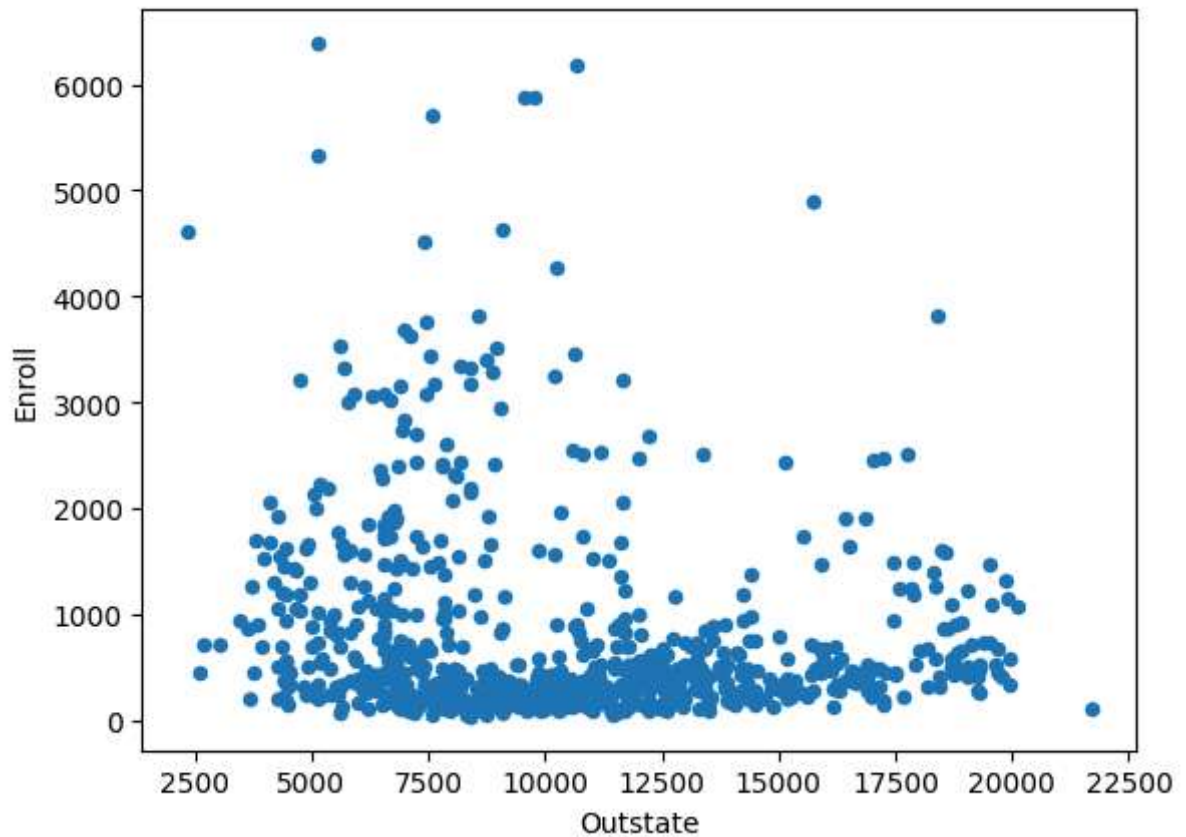
Interesting, this is similar to what I was expecting, but I thought the relationship would be more linear. Let's see if the proportion of PhD professors affects graduation rate.

```
In [18]: college.plot.scatter(x='PhD', y='Grad.Rate');
```

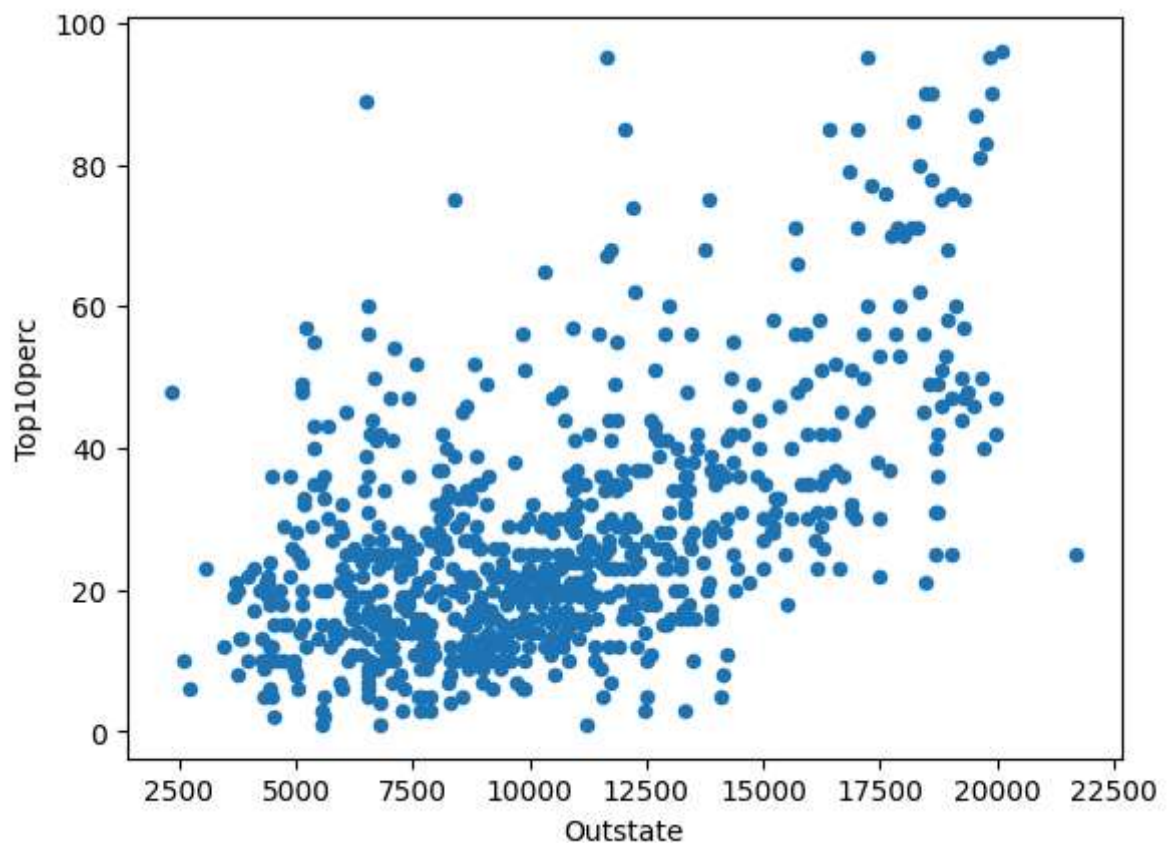


Again, this is interesting. There is no visible, meaningful relationship between graduation rate and the proportion of PhD professors. I actually expected this to be slightly inversely proportional because PhD professors typically run labs and perform their own research, so they may be less invested in teaching. I want to explore the effect of out of state tuition on enrollment now.

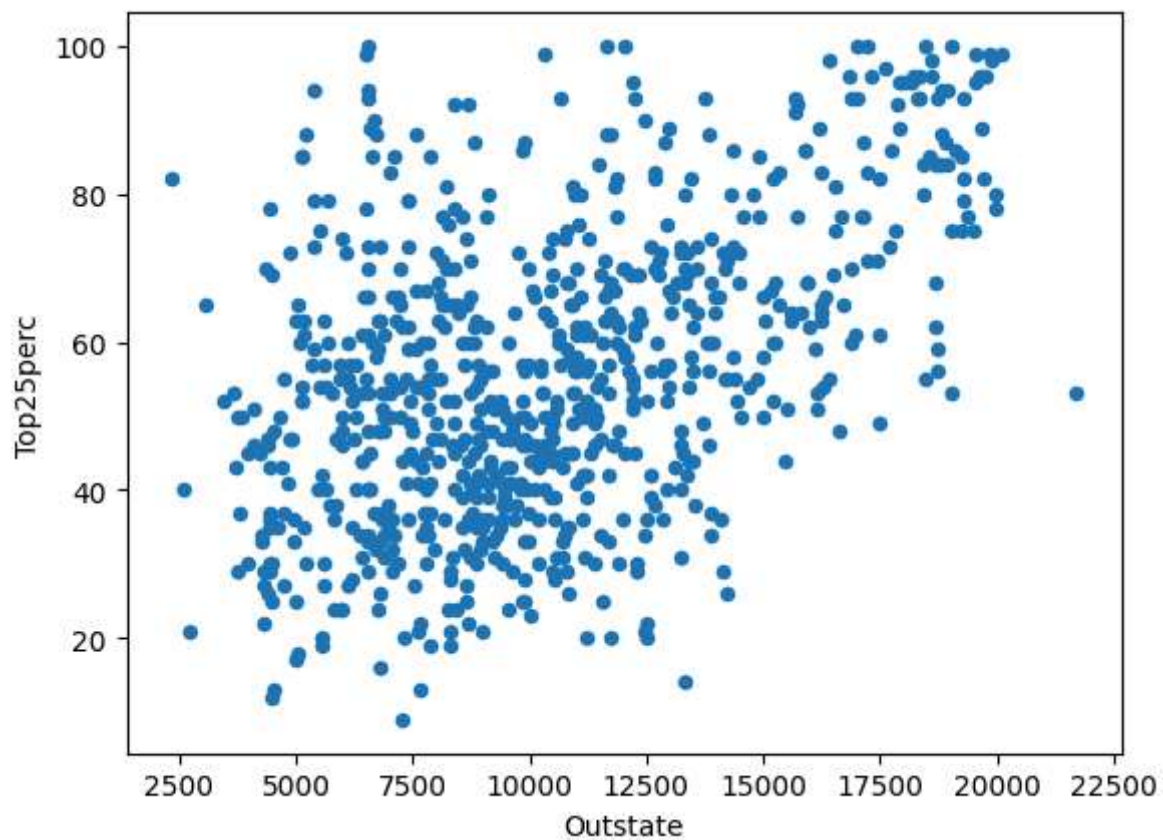
```
In [19]: college.plot.scatter(x='Outstate', y='Enroll');
```



```
In [20]: college.plot.scatter(x='Outstate', y='Top10perc');
```

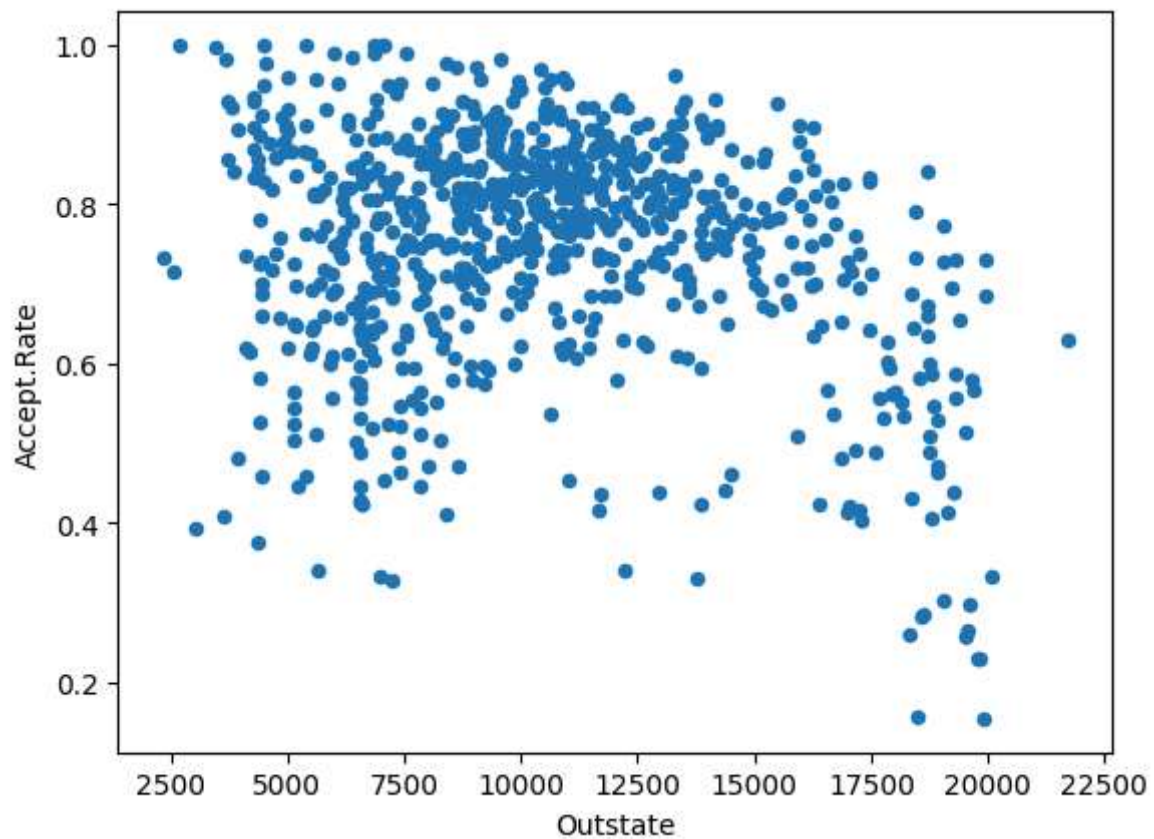


```
In [21]: college.plot.scatter(x='Outstate', y='Top25perc');
```

```
In [22]: college['Accept.Rate'] = college['Accept']/college['Apps']
```

```
In [23]: college.plot.scatter(x='Outstate', y='Accept.Rate');
```



There were several trends that I was interested in exploring, and most of them behaved as expected.

- The first relationship, Outstate vs Enroll, did not show a clear trend. I was expecting this to be negatively correlated, but I didn't adjust for the total number of students at the institution, so this result is conflated with other factors.
- The second relationship, Outstate vs Top10perc, was moderately positively correlated, which is what I was expecting. As with any product or service, more expensive is generally considered more premium or better, so I expected expensive institutions to attract the best highschool applicants.
- The third relationship, Outstate vs Top25perc, was also moderately positively correlated, for the same reason outlined in my Top10perc analysis.
- The final relationship, Outstate vs Acceptance Rate, was slightly negatively correlated, as expected. The more expensive the institution, the more selective it might be.

Problem 9

This exercise involves the Auto data set studied in the lab. Make sure that the missing values have been removed from the data.

```
In [24]: # Before getting started, identify and remove the missing data.
# From looking at the csv, I found that horsepower has missing values.
# These are question marks in the dataset, so we will drop that as well.
data_path = './data/Auto.csv'
auto = pd.read_csv(data_path)
auto.dropna(axis=0, how='any')
auto = auto[auto['horsepower'] != '?']
```

```
In [25]: # Origin also needs to be remapped. It is currently using a numerical representatio
# to describe the region of origin, which is not ideal. Let's resolve this.

origin_map = {1:'American',
              2:'European',
              3:'Japanese'}

remapped = []

for region in auto['origin']:
    remapped.append(origin_map[region])

auto['origin'] = pd.Series(remapped)
```

Part A

Which of the predictors are quantitative, and which are qualitative?

```
In [26]: quantitative = auto.describe().columns
```

```
qualitative = [column for column in auto.columns if column not in quantitative]
```

```
In [27]: quantitative
```

```
Out[27]: Index(['mpg', 'cylinders', 'displacement', 'weight', 'acceleration', 'year'], dtype='object')
```

```
In [28]: qualitative
```

```
Out[28]: ['horsepower', 'origin', 'name']
```

Hmmmmm, this doesn't match what I was expecting. Let's look at the type of each column.

```
In [29]: for column in auto.columns:
          print(column, auto[column].dtype)
```

```
mpg float64
cylinders int64
displacement float64
horsepower object
weight int64
acceleration float64
year int64
origin object
name object
```

Okay, so here is the root of the issue. Let's go through and change the types of the columns that should be quantitative.

```
In [30]: auto['horsepower'] = auto['horsepower'].astype('int64')
```

Now the steps from before can be repeated.

```
In [31]: quantitative = auto.describe().columns
          qualitative = [column for column in auto.columns if column not in quantitative]
```

```
In [32]: quantitative
```

```
Out[32]: Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
               'acceleration', 'year'],
              dtype='object')
```

```
In [33]: qualitative
```

```
Out[33]: ['origin', 'name']
```

Part B

What is the range of each quantitative predictor? You can answer this using the min() and max() methods in numpy.

```
In [34]: for column in quantitative:
          print(f'Range of {column}: {min(auto[column])} - {max(auto[column])}')
```

```
Range of mpg: 9.0 - 46.6
Range of cylinders: 3 - 8
Range of displacement: 68.0 - 455.0
Range of horsepower: 46 - 230
Range of weight: 1613 - 5140
Range of acceleration: 8.0 - 24.8
Range of year: 70 - 82
```

Part C

What is the mean and standard deviation of each quantitative predictor?

```
In [35]: for column in quantitative:
          print(f"{column} Mean: {round(auto[column].mean(), 2)}\n{column} Standard Devia
```

```
mpg Mean: 23.45
mpg Standard Deviation: 7.81

cylinders Mean: 5.47
cylinders Standard Deviation: 1.71

displacement Mean: 194.41
displacement Standard Deviation: 104.64

horsepower Mean: 104.47
horsepower Standard Deviation: 38.49

weight Mean: 2977.58
weight Standard Deviation: 849.4

acceleration Mean: 15.54
acceleration Standard Deviation: 2.76

year Mean: 75.98
year Standard Deviation: 3.68
```

Part D

Now remove the 10th through 85th observations. What is the range, mean, and standard deviation of each predictor in the subset of the data that remains?

```
In [36]: len(auto)
```

```
Out[36]: 392
```

```
In [37]: auto_subset = auto.drop(auto.index[10:85], axis = 0)
```

```
In [38]: len(auto_subset)
```

Out[38]: 317

```
In [39]: for column in quantitative:
          print(f'Range of {column}: {min(auto_subset[column])} - {max(auto_subset[column])}')
```

Range of mpg: 11.0 - 46.6
 Range of cylinders: 3 - 8
 Range of displacement: 68.0 - 455.0
 Range of horsepower: 46 - 230
 Range of weight: 1649 - 4997
 Range of acceleration: 8.5 - 24.8
 Range of year: 70 - 82

```
In [40]: for column in quantitative:
          print(f'{column} Mean: {round(auto_subset[column].mean(), 2)}\n{column} Standard Deviation: {round(auto_subset[column].std(), 2)}')
```

mpg Mean: 24.37
 mpg Standard Deviation: 7.87

 cylinders Mean: 5.38
 cylinders Standard Deviation: 1.66

 displacement Mean: 187.88
 displacement Standard Deviation: 100.17

 horsepower Mean: 101.0
 horsepower Standard Deviation: 36.0

 weight Mean: 2938.85
 weight Standard Deviation: 811.64

 acceleration Mean: 15.7
 acceleration Standard Deviation: 2.72

 year Mean: 77.12
 year Standard Deviation: 3.13

Part E

Using the full data set, investigate the predictors graphically, using scatterplots or other tools of your choice. Create some plots highlighting the relationships among the predictors. Comment on your findings.

```
In [41]: fig, axes = plt.subplots(2, 2, figsize=(15,10));

          auto['mpg'].hist(ax=axes[0,0],bins = 10);
          axes[0,0].set_title('mpg');
          axes[0,0].set_xlabel('Value');
          axes[0,0].set_ylabel('Frequency');

          auto['cylinders'].hist(ax=axes[0,1],bins = 5);
          axes[0,1].set_title('cylinders');
          axes[0,1].set_xlabel('Value');
```

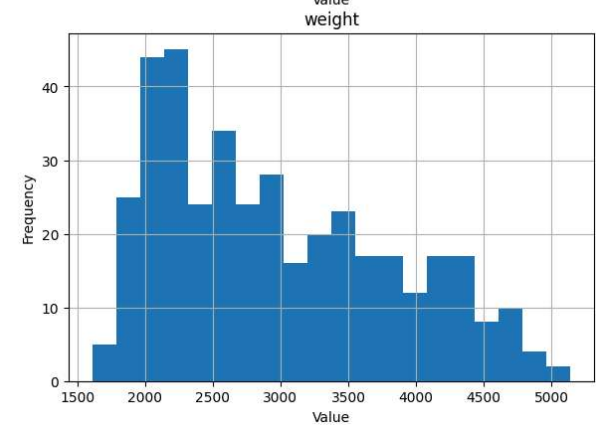
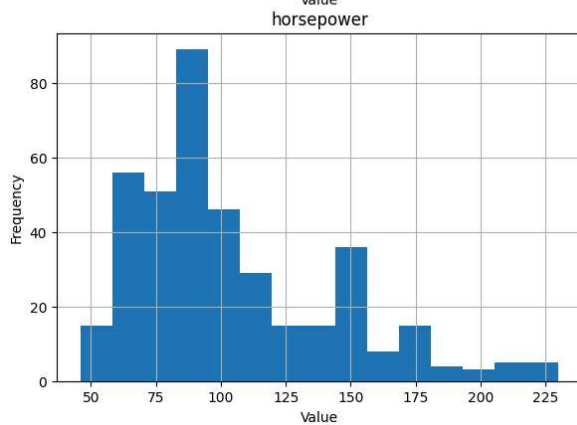
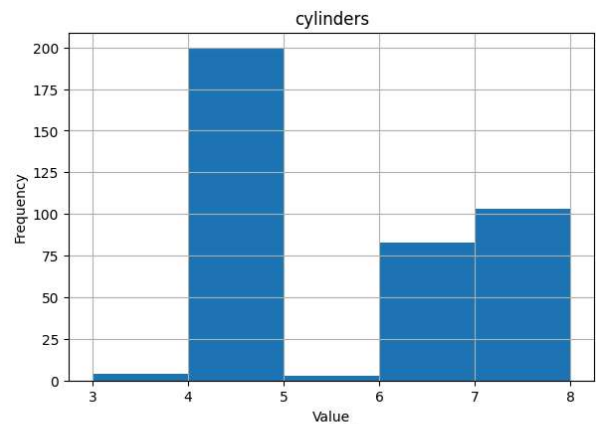
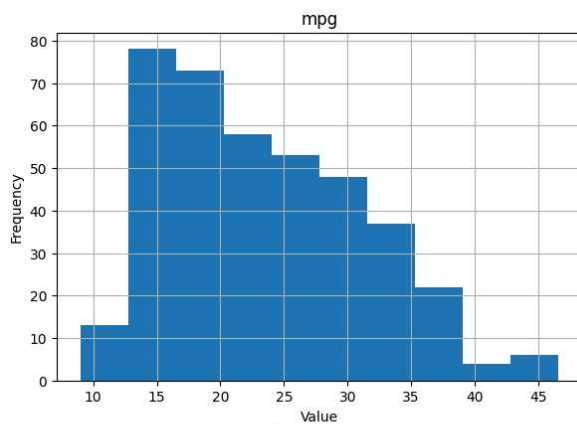
```

axes[0,1].set_ylabel('Frequency');

auto['horsepower'].hist(ax=axes[1,0],bins = 15);
axes[1,0].set_title('horsepower');
axes[1,0].set_xlabel('Value');
axes[1,0].set_ylabel('Frequency');

auto['weight'].hist(ax=axes[1,1],bins = 20);
axes[1,1].set_title('weight');
axes[1,1].set_xlabel('Value');
axes[1,1].set_ylabel('Frequency');

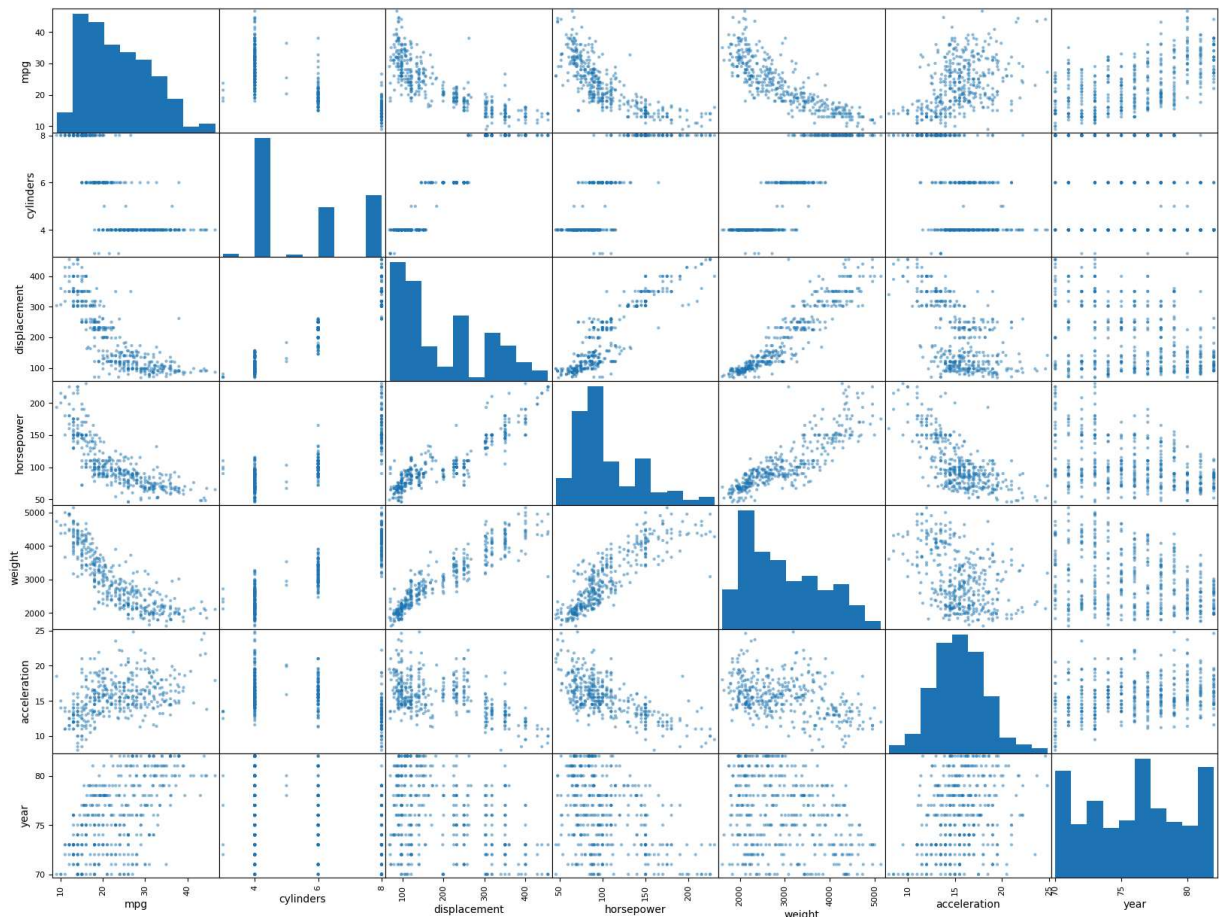
```



```

In [42]: pd.plotting.scatter_matrix(auto[quantitative], figsize=(20, 15));

```



There are several interesting trends in these variables. Most notably, displacement vs horsepower, mpg vs weight, and mpg vs year.

- Displacement vs Horsepower:
 - This is nearly a perfectly linear, positively correlated variable.
- MPG vs Weight:
 - This appears to have a relationship that resembles exponential decay. MPG is inversely proportional to Weight.
- MPG vs Year:
 - MPG and Year appear to be moderately positively correlated, which is likely due to design improvements and efficiency regulations.

Part F

Suppose that we wish to predict gas mileage (mpg) on the basis of the other variables. Do your plots suggest that any of the other variables might be useful in predicting mpg? Justify your answer.

The plots, above, suggest displacement, horsepower, and weight are strong predictors of gas mileage, while acceleration is a weak predictor. The strong predictors are all negatively correlated with MPG, and each relationship can be categorized as

'exponential decay.' Acceleration time is positively correlated with mpg, which is an intuitive relationship based on how acceleration is correlated with the other factors.

Problem 10

This exercise involves the Boston housing data set.

Part A

To begin, load in the Boston data set, which is part of the ISLP library.

```
In [45]: from ISLP import load_data
```

```
In [46]: boston = load_data('Boston')
```

Part B

How many rows are in this data set? How many columns? What do the rows and columns represent?

```
In [49]: boston.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 13 columns):
#   Column      Non-Null Count  Dtype
---  -
0   crim        506 non-null    float64
1   zn           506 non-null    float64
2   indus        506 non-null    float64
3   chas         506 non-null    int64
4   nox          506 non-null    float64
5   rm           506 non-null    float64
6   age          506 non-null    float64
7   dis          506 non-null    float64
8   rad          506 non-null    int64
9   tax          506 non-null    int64
10  ptratio      506 non-null    float64
11  lstat        506 non-null    float64
12  medv         506 non-null    float64
dtypes: float64(10), int64(3)
memory usage: 51.5 KB
```

From the ISLP website for this dataset (<https://intro-stat-learning.github.io/ISLP/datasets/Boston.html>)

crim: per capita crime rate by town.

zn: proportion of residential land zoned for lots over 25,000 sq.ft.

indus: proportion of non-retail business acres per town.

chas: Charles River dummy variable (= 1 if tract bounds river; 0 otherwise).

nox: nitrogen oxides concentration (parts per 10 million).

rm: average number of rooms per dwelling.

age: proportion of owner-occupied units built prior to 1940.

dis: weighted mean of distances to five Boston employment centres.

rad: index of accessibility to radial highways.

tax: full-value property-tax rate per \$10,000.

ptratio: pupil-teacher ratio by town.

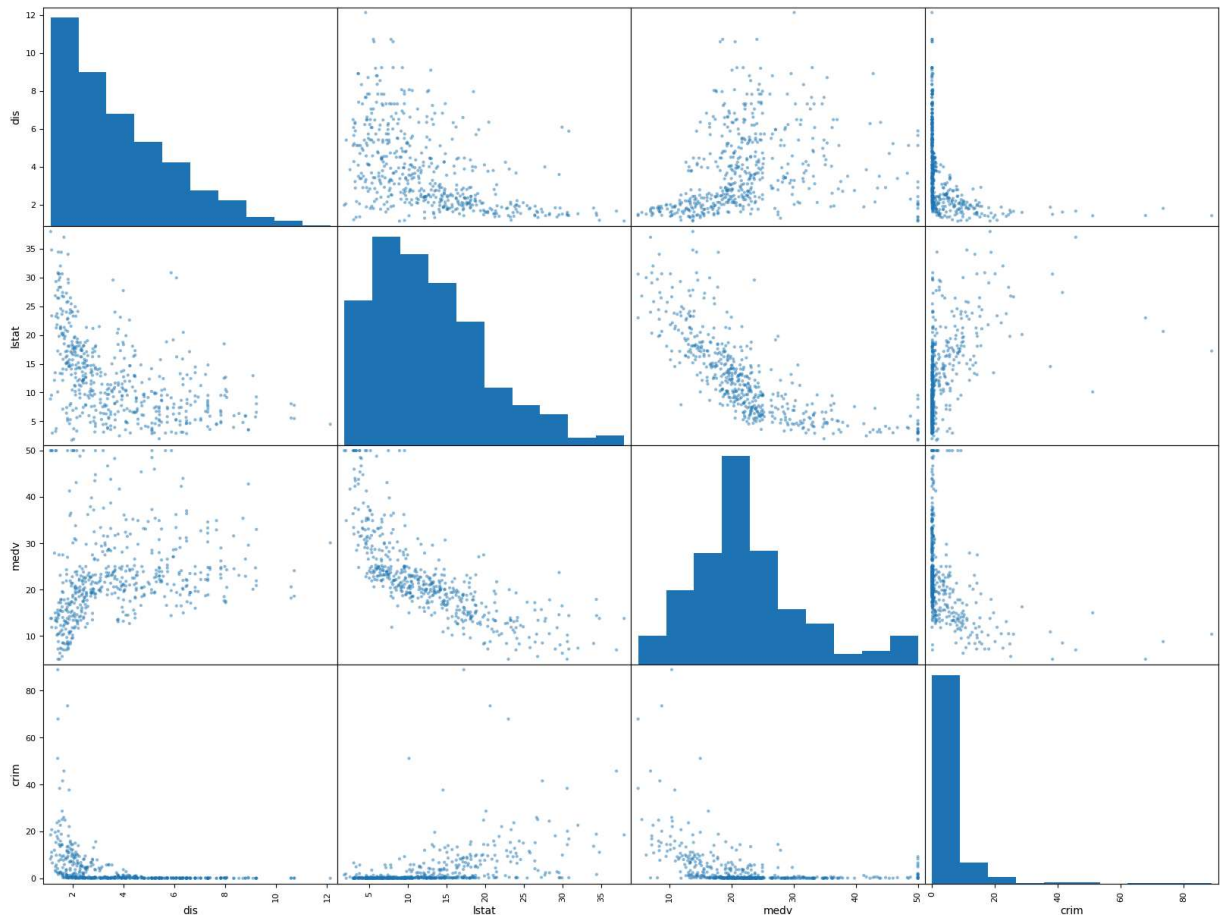
lstat: lower status of the population (percent).

medv: median value of owner-occupied homes in \$1000s.

Part C

Make some pairwise scatterplots of the predictors (columns) in this data set. Describe your findings.

```
In [53]: pd.plotting.scatter_matrix(boston[['dis', 'lstat', 'medv', 'crim']], figsize=(20, 1
```



These columns were selected because my hypothesis was the weighted mean of distances to five Boston employment centers would be correlated with the other predictors. I found that there were correlations with the other variables, but not what I expected.

- Dis vs Lstat:
 - This was negatively, exponentially related. I expected positive, linear correlation.
- Dis vs Medv:
 - This was positively, logarithmically related. I expected negative, linear correlation.
- Dis vs Crim:
 - This was negatively, exponentially related. I expected negative, linear correlation.

The key takeaway from my poor assumption is that population density is probably the driving variable.

Part D

Are any of the predictors associated with per capita crime rate? If so, explain the relationship.

As seen in the previous scatter matrix, 'dis' is associated with crime rate. My assumption is that 'dis' is actually a strong predictor for population density.

Part E

Do any of the suburbs of Boston appear to have particularly high crime rates? Tax rates? Pupil-teacher ratios? Comment on the range of each predictor.

```
In [75]: boston['crim'].sort_values(ascending=False).head(10)
```

```
Out[75]: 380    88.9762
         418    73.5341
         405    67.9208
         410    51.1358
         414    45.7461
         404    41.5292
         398    38.3518
         427    37.6619
         413    28.6558
         417    25.9406
         Name: crim, dtype: float64
```

```
In [58]: boston['tax'].sort_values(ascending=False).head(10)
```

```
Out[58]: 492    711
         491    711
         490    711
         489    711
         488    711
         405    666
         452    666
         456    666
         455    666
         454    666
         Name: tax, dtype: int64
```

```
In [59]: boston['ptratio'].sort_values(ascending=False).head(10)
```

```
Out[59]: 354    22.0
         355    22.0
         135    21.2
         127    21.2
         136    21.2
         141    21.2
         139    21.2
         138    21.2
         137    21.2
         140    21.2
         Name: ptratio, dtype: float64
```

```
In [76]: for column in boston.columns:
         print(f'Mean {column} Value: {boston[column].mean()}\tMedian {column} Value: {b
```

Mean crim Value: 3.613523557312254	Median crim Value: 0.25651
Mean zn Value: 11.363636363636363	Median zn Value: 0.0
Mean indus Value: 11.13677865612648	Median indus Value: 9.69
Mean chas Value: 0.0691699604743083	Median chas Value: 0.0
Mean nox Value: 0.5546950592885376	Median nox Value: 0.538
Mean rm Value: 6.284634387351779	Median rm Value: 6.2085
Mean age Value: 68.57490118577076	Median age Value: 77.5
Mean dis Value: 3.795042687747036	Median dis Value: 3.2074499999999997
Mean rad Value: 9.549407114624506	Median rad Value: 5.0
Mean tax Value: 408.2371541501976	Median tax Value: 330.0
Mean ptratio Value: 18.455533596837945	Median ptratio Value: 19.05
Mean lstat Value: 12.653063241106722	Median lstat Value: 11.36
Mean medv Value: 22.532806324110677	Median medv Value: 21.2

There does not appear to be any overlap in the extremes of each of the predictors. That said, there is a clear divergence between mean and median values for crim, indus, and age variables, suggesting that they may be skewed by a small subset of suburbs with disproportionately high or low values for these variables.

Part F

How many of the suburbs in this data set bound the Charles river?

```
In [60]: len(boston[boston['chas'] == 1])
```

```
Out[60]: 35
```

35 of the suburbs are bound to the Charles River.

Part G

What is the median pupil-teacher ratio among the towns in this data set?

```
In [61]: boston['ptratio'].median()
```

```
Out[61]: 19.05
```

Part H

Which suburb of Boston has lowest median value of owner-occupied homes? What are the values of the other predictors for that suburb, and how do those values compare to the overall ranges for those predictors? Comment on your findings.

```
In [62]: boston['medv'].sort_values().head(1)
```

```
Out[62]: 398    5.0
         Name: medv, dtype: float64
```

```
In [63]: boston.iloc[398]
```

```
Out[63]: crim      38.3518
         zn        0.0000
         indus     18.1000
         chas      0.0000
         nox       0.6930
         rm        5.4530
         age      100.0000
         dis       1.4896
         rad       24.0000
         tax      666.0000
         ptratio   20.2000
         lstat     30.5900
         medv      5.0000
         Name: 398, dtype: float64
```

```
In [70]: for column in boston.columns:
         print(f'Minimum {column} Value: {min(boston[column])}\t\tMaximum {column} Value: {max(boston[column])}\t\tMean {column} Value: {boston[column].mean()}')
```

Minimum crim Value: 0.00632 Maximum crim Value: 88.9762 Mean crim Value: 3.613523557312254

Minimum zn Value: 0.0 Maximum zn Value: 100.0 Mean zn Value: 11.363636363636363

Minimum indus Value: 0.46 Maximum indus Value: 27.74 Mean indus Value: 11.13677865612648

Minimum chas Value: 0 Maximum chas Value: 1 Mean chas Value: 0.0691699604743083

Minimum nox Value: 0.385 Maximum nox Value: 0.871 Mean nox Value: 0.5546950592885376

Minimum rm Value: 3.561 Maximum rm Value: 8.78 Mean rm Value: 6.284634387351779

Minimum age Value: 2.9 Maximum age Value: 100.0 Mean age Value: 68.57490118577076

Minimum dis Value: 1.1296 Maximum dis Value: 12.1265 Mean dis Value: 3.795042687747036

Minimum rad Value: 1 Maximum rad Value: 24 Mean rad Value: 9.549407114624506

Minimum tax Value: 187 Maximum tax Value: 711 Mean tax Value: 408.2371541501976

Minimum ptratio Value: 12.6 Maximum ptratio Value: 22.0 Mean ptratio Value: 18.455533596837945

Minimum lstat Value: 1.73 Maximum lstat Value: 37.97 Mean lstat Value: 12.653063241106722

Minimum medv Value: 5.0 Maximum medv Value: 50.0 Mean medv Value: 22.532806324110677

These predictors tend to be on the worse end of the range for suburb 398. It has relatively high crime, low number of rooms per dwelling, and a high value for lower status of the population. Each of these values was compared to the mean of all suburbs.

Part I

In this data set, how many of the suburbs average more than seven rooms per dwelling? More than eight rooms per dwelling? Comment on the suburbs that average more than

eight rooms per dwelling.

```
In [72]: len(boston[boston['rm']>7])
```

```
Out[72]: 64
```

```
In [74]: boston[boston['rm']>8]
```

```
Out[74]:
```

	crim	zn	indus	chas	nox	rm	age	dis	rad	tax	ptratio	lstat	medv
97	0.12083	0.0	2.89	0	0.4450	8.069	76.0	3.4952	2	276	18.0	4.21	38.7
163	1.51902	0.0	19.58	1	0.6050	8.375	93.9	2.1620	5	403	14.7	3.32	50.0
204	0.02009	95.0	2.68	0	0.4161	8.034	31.9	5.1180	4	224	14.7	2.88	50.0
224	0.31533	0.0	6.20	0	0.5040	8.266	78.3	2.8944	8	307	17.4	4.14	44.8
225	0.52693	0.0	6.20	0	0.5040	8.725	83.0	2.8944	8	307	17.4	4.63	50.0
226	0.38214	0.0	6.20	0	0.5040	8.040	86.5	3.2157	8	307	17.4	3.13	37.6
232	0.57529	0.0	6.20	0	0.5070	8.337	73.3	3.8384	8	307	17.4	2.47	41.7
233	0.33147	0.0	6.20	0	0.5070	8.247	70.4	3.6519	8	307	17.4	3.95	48.3
253	0.36894	22.0	5.86	0	0.4310	8.259	8.4	8.9067	7	330	19.1	3.54	42.8
257	0.61154	20.0	3.97	0	0.6470	8.704	86.9	1.8010	5	264	13.0	5.12	50.0
262	0.52014	20.0	3.97	0	0.6470	8.398	91.5	2.2885	5	264	13.0	5.91	48.8
267	0.57834	20.0	3.97	0	0.5750	8.297	67.0	2.4216	5	264	13.0	7.44	50.0
364	3.47428	0.0	18.10	1	0.7180	8.780	82.9	1.9047	24	666	20.2	5.29	21.9

These suburbs appear to be wealthier and safer based on lstat, medv, and crim. Surprisingly, these wealthy suburbs also have some of the lowest tax rates of all suburbs, which is the opposite of what I would expect.